



Aqueduct Materials (aqueducts)

Aquädukte werden aus einem von N römischen Materialien gebaut. Das i -te Material hat eine Stärke S_i und einen Preis P_i .

Wir sagen, das i -te Material *dominiert* das j -te Material genau dann, wenn:

- es stärker ist ($S_i > S_j$) und
- es billiger ist ($P_i < P_j$).

Der Senat will die Nutzung von **einem** Material einschränken, sodass es ausschliesslich für Rom verwendet wird, um dessen einzigartiges Aussehen zu gewährleisten. Die $N - 1$ anderen Materialien sind für den Rest des Reiches *verfügbar*.

Ein Material ist *optimal* für den Rest des Reiches, wenn:

- es verfügbar ist und
- es nicht von einem anderen verfügbaren Material dominiert wird.

Für jedes k , angenommen, das k -te Material sei für Rom reserviert, zähle die Zahl C_k der optimalen Materialien für den Rest des Reiches.

Implementierung

Du musst eine einzelne `.cpp` Quelldatei einreichen.



Unter den Anhängen zu dieser Aufgabe findest du eine Vorlage `aqueducts.cpp` mit einer Beispielimplementierung.

Du musst die folgende Funktion implementieren:

C++

```
vector<int> count(int N, vector<int> S, vector<int> P)
```

- Die Ganzzahl N stellt die Anzahl der Materialien dar.
- Das Array S , indiziert von 0 bis $N - 1$, enthält die Werte $S_0, S_1, \dots, S_{N-1}$.
- Das Array P , indiziert von 0 bis $N - 1$, enthält die Werte $P_0, P_1, \dots, P_{N-1}$.
- Die Funktion soll ein Array C zurückgeben, welches die Werte $C_0, C_1, \dots, C_{N-1}$ enthält, wobei C_k die Zahl der für den Rest des Reiches übrigbleibenden Materialien ist, wenn das k -te Material für Rom reserviert ist.

Beispielgrader

Das Verzeichnis der Aufgabe enthält eine vereinfachte Version des Jury-Graders, die du verwenden kannst, um deine Lösung lokal zu testen. Der vereinfachte Grader liest die Eingangsdaten aus der Datei `stdin`, ruft die Funktion `count` auf und schreibt schliesslich die Ausgabe in die Datei `stdout` mit folgendem Format.

Die Eingabe besteht aus $N + 1$ Zeilen, die Folgendes enthalten:

- Zeile 1: die Ganzzahl N .
- Zeile $2 + i$ ($0 \leq i < N$): die Ganzzahlen S_i und P_i .

Die Ausgabe besteht aus einer einzigen Zeile, die den von der Funktion `count` zurückgegebenen Wert enthält.

Einschränkungen

- $2 \leq N \leq 300\,000$.
- $1 \leq S_i, P_i \leq 1\,000\,000$.
- Für alle $0 \leq i < j \leq N - 1$, haben wir $S_i \neq S_j$ und $P_i \neq P_j$.

Punktevergabe

Dein Programm wird anhand einer Reihe von Testfällen getestet, die nach Teilaufgaben gruppiert sind. Um die Punktzahl zu erhalten, die einer Teilaufgabe zugeordnet ist, musst du alle darin enthaltenen Testfälle korrekt lösen.

- **Teilaufgabe 0 [0 Punkte]**: Beispiel-Testfälle.
- **Teilaufgabe 1 [13 Punkte]**: $N \leq 100$.
- **Teilaufgabe 2 [22 Punkte]**: $N \leq 5000$.
- **Teilaufgabe 3 [28 Punkte]**: $|S_i - P_i| \leq 2$.
- **Teilaufgabe 4 [37 Punkte]**: Keine zusätzlichen Einschränkungen.

Beispiele

stdin	stdout
3 10 20 30 10 20 30	1 2 1
9 10 11 4 6 11 13 14 7 1 4 6 10 9 2 12 12 3 5	2 2 2 3 2 2 4 2 2

Erklärung

Erklärung des ersten Beispiels.

- Wenn $k = 0$, ist Material 1 optimal.
- Wenn $k = 1$, sind Materialien 0 und 2 optimal.
- Wenn $k = 2$, ist Material 1 optimal.

Erklärung des zweiten Beispiels.

- Wenn $k = 0$, sind Materialien 3 und 6 optimal.
- Wenn $k = 1$, sind Materialien 3 und 6 optimal.
- Wenn $k = 2$, sind Materialien 3 und 6 optimal.
- Wenn $k = 3$, sind Materialien 0, 6 und 7 optimal.
- Wenn $k = 4$, sind Materialien 3 und 6 optimal.
- Wenn $k = 5$, sind Materialien 3 und 6 optimal.
- Wenn $k = 6$, sind Materialien 1, 3, 4 und 8 optimal.
- Wenn $k = 7$, sind Materialien 3 und 6 optimal.
- Wenn $k = 8$, sind Materialien 3 und 6 optimal.